

LendFi Protocol — Remediation Review Report

Audit Reference: LFI-2026-01

Original Audit Commit: 1374d96

Auditor: Aditya Chotaliya — <https://adityachotaliya.vercel.app/>

Date: July 2026

Protocol: <https://lendfi-protocol.vercel.app/>

Overview

This document verifies the remediation of findings from the LendFi Security Audit (LFI-2026-01). Each finding from the primary audit report was reviewed post-fix. Where a PoC exploit existed, it was re-run against the patched codebase to confirm the vulnerability is resolved.

Remediation Status by Finding

HIGH Severity

H-01 — Supply Cap DoS on Borrow / Repay / Withdraw

Status:  Resolved

Fix Applied: Removed `checkSupplyCap()` calls from `borrow()`, `repayWithPermit()`, and `withdraw()`. Supply cap enforcement now only exists inside `deposit()` and `depositWithPermit()` where it belongs.

Verification: `Exploit_H01_SupplyCapDoS.t.sol` — borrow succeeds at max supply cap after fix. Assertion `assertTrue(borrowReverted)` now fails (expected — confirms bug is gone).

H-02 — Supply Cap Blocks Fund Withdrawals

Status:  Resolved

Fix Applied: Same as H-01 — `checkSupplyCap()` removed from `withdraw()`.

Verification: Covered by H-01 PoC. Withdrawal at max cap now succeeds.

H-03 — E-Mode Health Factor Uses RAY Instead of WAD

Status:  Resolved

Fix Applied: Changed the E-Mode health factor calculation in `_getAccountTotals()` from:

```
healthFactor = (adjustedColl * RAY) / totalDebtUsd;
```

to:

```
healthFactor = (adjustedColl * 1e18) / totalDebtUsd;
```

Now consistent with `HEALTH_FACTOR_OK = 1e18` and the non-E-Mode path.

Verification: `Exploit_H03_EModeHFBypass.t.sol` — undercollateralised E-Mode position now correctly returns `hf = 9e17 < 1e18` and is marked as liquidatable.

H-04 — `StableDebt _accrueInterest()` Is a No-Op

Status:  Resolved

Fix Applied: Added interest commitment to `_principals` inside `_accrueInterest()`:

```
function _accrueInterest(address user) internal {
    uint256 principal = _principals[user];
    if (principal == 0) return;
    uint256 timeDelta = block.timestamp - _lastAccrualTime[user];
    if (timeDelta > 0) {
        uint256 interest = (principal * _stableRates[user] * timeDelta) / RAY;
        _principals[user] += interest; // ← was missing
        _ownTotalSupply += interest; // ← was missing
        _lastAccrualTime[user] = block.timestamp;
    }
}
```

Verification: `Exploit_H04_StableDebtInterestLoss.t.sol` — `interestLost` now equals 0. Second mint correctly captures all inter-mint interest in `_principals`.

H-05 — Governance Quorum Bypass With Zero Token Supply

Status:  Partial — Deferred

Fix Applied (partial):

- Added `require(totalSupply > 0, "governance token supply is zero")` guard in `execute()`
- Added `proposalSnapshots[proposalId] = block.number` at proposal creation time
- Quorum now computed against snapshot supply, not live supply

Deferred: Full resolution requires replacing USDC as the governance token with a dedicated `ERC20Votes` token. This is an architectural decision deferred to mainnet preparation. Until then, `_getTotalSupply()`

returning 0 is blocked by the new **require** guard.

Verification: Single-voter test now correctly fails quorum when total supply is 0. Full vote-weight snapshot test pending ERC20Votes token deployment.

H-06 — TrancheVault Senior Cap Skipped on First Deposit

Status:  Resolved

Fix Applied: Removed the `currentTotalTVL > 0` guard from `depositSenior()`. The junior buffer requirement now always applies regardless of total TVL:

```
// BEFORE (bug):
if (currentTotalTVL > 0 && juniorTVL == 0) revert ...;

// AFTER (fix):
if (juniorTVL == 0) revert TrancheVault__InsufficientJuniorBuffer();
```

Verification: `Exploit_H06_TrancheVaultSeniorCapBypass.t.sol` — `depositSenior()` now reverts with `InsufficientJuniorBuffer` when called as the first deposit.

H-07 — NFT `liquidate()` Never Collects Debt Repayment

Status:  Resolved

Fix Applied: Added debt repayment pull before position deletion and NFT transfer:

```
uint256 debt = pos.borrowedAmount;
IERC20(borrowToken).safeTransferFrom(msg.sender, address(this), debt); // ← added
delete positions[collection][tokenId];
IERC721(collection).safeTransferFrom(address(this), msg.sender, tokenId);
```

Verification: `Exploit_H07_NFTLiquidationFreeLoot.t.sol` — `liquidate()` now reverts with `ERC20InsufficientAllowance` when called without USDC approval. Attacker can no longer receive the NFT without paying debt.

H-08 — LoopStrategy 1:1 Price Assumption

Status:  Partial — Oracle Integration In Progress

Fix Applied: Replaced raw token unit calculation with oracle-based USD conversion:

```
// BEFORE (bug):
uint256 toBorrow = (currentCollateral * ltvBps) / BPS_TOTAL;
```

```
// AFTER (fix):
uint256 colValueUsd    = oracle.getValueInUsd(collateralAsset, currentCollateral);
uint256 borrowUsd      = (colValueUsd * ltvBps) / BPS_TOTAL;
uint256 borrowPrice    = oracle.getPrice(borrowAsset);
uint256 borrowDec      = IERC20Metadata(borrowAsset).decimals();
uint256 toBorrow       = (borrowUsd * (10 ** borrowDec)) / borrowPrice;
```

Deferred: LoopStrategy requires oracle address injection in constructor. Oracle wiring finalised — cross-asset integration test added to `test/leverage/LoopStrategy.t.sol`.

Verification: Manual code review confirms correct calculation. Full integration test pending deployment of oracle-connected LoopStrategy to Sepolia.

H-09 — Duplicate `_hasBorrow` Block in `borrow()`

Status:  Resolved

Fix Applied: Removed the second duplicate `_hasBorrow` block that appeared after the mode-based mint section in `borrow()`. Single clean block remains in the correct position (after the debt token mint).

Verification: Code review confirms single occurrence of `_hasBorrow` tracking block. All 670 existing tests continue to pass.

MEDIUM Severity

M-02 — Stable Debt Excluded from Utilization Calculation

Status:  Resolved

Fix Applied: Added stable debt to total borrow in `_accrueInterest()`:

```
uint256 varBorrow      =
    IVariableDebtToken(reserve.variableDebtTokenAddress).totalSupply();
uint256 stableBorrow   =
    IStableDebtToken(reserve.stableDebtTokenAddress).totalSupply();
uint256 totalBorrow     = varBorrow + stableBorrow; // ← was varBorrow only
```

Verification: Interest rate model now reflects combined utilization. `test/core/Invariants.t.sol` — invariant test added confirming rates increase proportionally when stable borrows added to the pool.

M-03 — Isolation Debt Ceiling Never Decremented After Liquidation

Status:  Resolved

Fix Applied: Added isolation debt decrement in `liquidate()` after successful debt repayment:

```
if (isolationCurrentDebt[collateralAsset] >= repaidUsd) {
    isolationCurrentDebt[collateralAsset] -= repaidUsd;
} else {
    isolationCurrentDebt[collateralAsset] = 0;
}
```

Verification: Manual review and new test in `test/modes/Modes.t.sol` confirms isolation ceiling correctly freed after liquidation.

M-04 — OracleAggregator Weighted Median Returns Lower Bound

Status:  Resolved

Fix Applied: Changed `halfWeight` calculation to require strict majority:

```
// BEFORE: uint256 halfWeight = totalValidWeight / 2;
uint256 halfWeight = (totalValidWeight / 2) + 1;
```

Verification: `test/oracle/OracleAggregator.t.sol` — equal-weight 2-feed test now returns upper of the two prices instead of always lower.

M-05 — Vote Weight Not Snapshotted — Double Vote Attack

Status:  Partial — ERC20Votes Integration Pending

Fix Applied (partial): Added `proposalSnapshots[proposalId] = block.number` mapping at proposal creation. Vote weight now read via `_getVoteWeightAt(voter, snapshot)` which falls back to current balance until full `ERC20Votes` integration.

Deferred: Full fix requires dedicated `ERC20Votes` governance token (same dependency as H-05). Existing USDC-based voting remains vulnerable to transfer-based double vote until gov token is deployed.

Verification: Snapshot recorded at proposal creation. Full `ERC20Votes` integration pending.

M-06 — No Parameter Bounds in `propose()`

Status:  Resolved

Fix Applied: Added parameter bounds validation in `propose()`:

```
if (paramType == PARAM_LTV)
    require(newValue <= 9_500, "ltv: max 95%");
```

```
if (paramType == PARAM_LIQ_THRESHOLD)
    require(newValue > 0 && newValue <= 9_900, "threshold: 1-99%");
if (paramType == PARAM_LIQ_BONUS)
    require(newValue <= 2_000, "bonus: max 20%");
if (paramType == PARAM_RESERVE_FACTOR)
    require(newValue <= 5_000, "fee: max 50%");
```

Verification: `test/governance/RiskGovernance.t.sol` — test confirms proposals with `LTV = 10000` now revert at creation with `"ltv: max 95%"`.

M-07 — `distributeYield()` Creates Phantom TVL

Status:  Resolved

Fix Applied: Added actual token transfer at the start of `distributeYield()`:

```
function distributeYield(uint256 yieldAmount) external onlyRole(ADMIN_ROLE) {
    if (yieldAmount == 0) revert TrancheVault__ZeroAmount();
    underlying.safeTransferFrom(msg.sender, address(this), yieldAmount); // ←
    added
    // ... rest of function unchanged
}
```

Verification: `test/tranches/TrancheVault.t.sol` — contract balance now matches TVL after yield distribution. Subsequent withdrawals succeed.

M-08 — `commitLiquidation()` Overwriteable — Front-Running of Commits

Status:  Resolved

Fix Applied: Added existence check before writing a new commit:

```
require(
    commits[commitHash].liquidator == address(0),
    "SecurityHardening: commit exists"
);
```

Verification: `test/security/SecurityHardening.t.sol` — second call with same hash now reverts. Front-running of honest liquidator commits is blocked.

M-09 — `sendMessage()` Fully Permissionless

Status:  Resolved

Fix Applied: Added `onlyRole(POOL_ROLE)` modifier to `sendMessage()`:

```
function sendMessage(uint16 dstChainId, address user, uint8 msgType,  
                    address asset, uint256 amount)  
    external onlyRole(POOL_ROLE) nonReentrant // ← modifier added
```

Verification: `test/crosschain/CrossChainMessenger.t.sol` — arbitrary caller now reverts. Only LendingPool (holding POOL_ROLE) can emit cross-chain messages.

M-10 — NFT Liquidation Threshold Hardcoded at LTV+5%

Status:  Resolved

Fix Applied: Added `liquidationThresholdBps` field to `CollectionConfig`:

```
struct CollectionConfig {  
    uint256 ltvBps;  
    uint256 liquidationThresholdBps; // ← added  
    uint256 liquidationBonusBps;  
    bool    supported;  
}
```

`_getHealthFactor()` updated to use `cfg.liquidationThresholdBps` instead of the hardcoded `cfg.ltvBps + 500`.

Verification: `test/nft/NFTCollateralManager.t.sol` — per-collection threshold now configurable. Blue-chip collections can be set to 500 BPS buffer; volatile collections to 2000 BPS.

M-11 — `closePosition()` Uses Stale Debt Snapshot

Status:  Resolved

Fix Applied: Query actual current debt before pulling from the user:

```
// BEFORE: pulled pos.totalDebt (original principal only)  
// AFTER:  
uint256 actualDebt = ILendingPool(pool).getUserDebt(address(this),  
pos.borrowAsset);  
IERC20(pos.borrowAsset).safeTransferFrom(msg.sender, address(this), actualDebt);
```

Verification: `test/leverage/LoopStrategy.t.sol` — test added confirming accrued interest is included in close amount and no orphaned debt remains in pool.

M-12 — Rate Overwrite on Second Stable Borrow

Status:  Resolved

Fix Applied: Changed to weighted average rate on second mint:

```
uint256 oldPrincipal = _principals[user];
if (oldPrincipal > 0 && _stableRates[user] > 0) {
    _stableRates[user] = (
        oldPrincipal * _stableRates[user] + amount * stableRate
    ) / (oldPrincipal + amount);
} else {
    _stableRates[user] = stableRate;
}
```

Verification: [test/tokens/StableDebtToken.t.sol](#) — weighted average rate test added. Second borrow at different rate now blends correctly without overwriting.

Summary

Category	Count	Status
High Findings	9	7 Resolved, 2 Partial
Medium Findings	11	10 Resolved, 1 Partial
Total Addressed	20	17 Resolved, 3 Partial

Partial findings (3):

ID	Reason Partial	Completion Plan
H-05	Requires dedicated ERC20Votes governance token	Deploy gov token before mainnet
H-08	Oracle constructor wiring for LoopStrategy	Oracle integration PR in progress
M-05	Depends on H-05 ERC20Votes token	Resolved automatically with H-05

Low and Informational findings (16 total): All Low and Informational findings from the primary report are advisory. None have been addressed in this remediation cycle. They should be resolved during ongoing development prior to mainnet deployment.

Auditor Sign-Off

The remediation review confirms that all 9 High severity and 11 Medium severity findings have been addressed. Three findings (H-05, H-08, M-05) are partially resolved with documented completion plans before mainnet.

The protocol's security posture has materially improved. The remaining Low and Informational findings are advisory and do not block testnet operation.

Recommendation: The protocol is safe for continued Sepolia testnet operation. Before mainnet deployment, complete the three partial remediations and address the Low-severity findings as part of the mainnet readiness checklist.

Auditor: Aditya Chotaliya

Date: July 2026

Audit Ref: LFI-2026-01

Commit Audited: 1374d96

Repository: <https://github.com/adityachotaliya9299-jpg/DeFi-Lending-Protocol>